Report Cascade Game Part B

Xiangdi (Allie) Hu 1524339
Qianbin Dong 1511935

Introduction

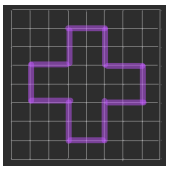
In this part of the program, we design and build a full agent program for playing the CASCADE game with hybrid decision-making strategy. During the placement phase, the agent is powered by a rule-based heuristic placement based on strategic insights of the game-play in this phase. During the play phase, we implement an optimized MINIMAX searching algorithm with alpha-beta pruning and iterative deepening for better adversarial performances and higher efficiency.

Description of our approach

First, for the Placement phase, we applied a logic-based evaluation placement strategy, which is derived by original game logic insights. The main body of implementation is in evaluation_placement.py and helper_placement.py.

Implementation of the strategy

There are four turns for each player. For the first turn, no matter which color we are representing, take one of the four centre coordinates (need to consider whether it is legal or not if we are BLUE (player at total turn 2)). For the second turn, if there are still possible legal centre coordinates, take it. Otherwise, switch to “safe-area” search. For the third and fourth turn, use “safe-area” search. For “safe-area” search: Filter all legal coordinates, evaluate them using the evaluation scoring function and choose the coordinate with the greatest score. (A special case is when three of the opponent stacks are already on different edges with considerable distances in between and even three different edges, then a good practice is to take the unoccupied edge/corner.) As a fallback, if the safe area is empty (all coordinates are illegal to place our new stack), we enlarge the searching area to the entire board.



(The area (coordinates) inside the purple frames is considered as the “safe area”, since it is close to the centre and far from the edge, suggesting high attack power and defense)

Reasons for choosing this algorithm

During the four turns for each player, the only possible movement is “PlaceAction”, which does not have significant interactive/adversarial effects on the board (it is only about coordinates taking), therefore, whether a state is preferable is clear and could be found by taking insights of the game plays (actual plays). Additionally, logic-based strategy saves our efforts for the complicated searching strategy in the play phase.

Explanation for the evaluation function

The placement evaluation function is designed to balance safety, central control, structure building, and future tactical potential. The agent first filters out dangerous coordinates, then scores the remaining positions using features related to attack, defence, alignment, and distance. This allows the agent to build a stable initial formation while preparing for future attacking opportunities and trying to reduce the power of the opponent in the latter phase.

The first feature “Triangle attack score” measures whether placing a new stack can create a triangle-like attacking formation with existing friendly stacks. A triangle structure can create multiple attacking angles/directions. The second feature “Pattern creation score” evaluates whether the new stack helps form at least one of the three useful strategic patterns (same color triangle, same color adjacency, and same color step) with friendly stacks (of our own color). These three patterns are very effective in terms of attacking, especially when performing CASCADE in the future. These two features are aiming to make it harder for the opponent to defend against all possible future threats and make the attack of our agent easier.

The third feature “Protection score” measures whether the new stack can protect the existing friendly stacks. Specifically, preventing the opponent stack from building a triangle structure in their next turn. This improves relatively long-term stability and reduces the chance of losing a stack in the near future after placement.
The fourth feature, “Same-line score”, checks whether our stack and the opponent's stacks are in the same row or column. This is useful because actions such as CASCADE depend strongly on directional alignment, so same-line positions may make future attack or defense more easier (especially within the safe area).
The last feature, “Nearest enemy distance score”, measures the distance to the nearest opponent stack, which helps balancing safety and pressure. To be more precise, being too close to the enemy may be dangerous, while being too far may reduce future interactions.
For the play phase, we use the Minimax algorithm with alpha-beta pruning, implemented in search.py and evaluation_play.py.

Implementation of the strategy

In each turn, our agent generates all legal actions from the current board state (regarded as the root state of the tree built in the searching process), including MOVE, EAT and CASCADE, and explores them with a depth-limited Minimax search. We treat our player as MAX and the opponent as MIN (let it also be optimal). The maximum search depth is fixed at 4. At each step, we copy the given board state, apply the legal action with the highest preferable score, and keep increasing the search depth and perform searching with each depth to obtain the best action on this level, and, therefore, update the overall best action until the depth limit or a terminal condition has been reached. We also implement alpha-beta pruning during the search. If a branch cannot improve the final result, we stop exploring it.
To be specific, we stop the search according to both the game rules, the pre-fixed depth limit, and the time limit. In any of these cases, we would stop the recursion and select and return the best action with the greatest evaluation value/score and its score we are able to find so far or compute an emergency action.

Reasons for choosing this algorithm

The main reason we decide and are able to choose and implement a Minimax with alpha-beta pruning algorithm for this phase is the nature of this Cascade game. This turn-based, adversarial game only involves 2 players, which is applicable for switching between the “MAX” and “MIN” levels. In addition, the board game is deterministic, observable and perfect-informed, which means we are able to evaluate a board configuration statically using an EVAL and perform the algorithm with the core idea of “maximising our utility while assuming the opponent is minimising our utility”. Moreover, according to the tactical characteristics of the action MOVE and CASCADE, increasing a few steps of looking-forwards, would be very helpful and reliable (compared to long-term playouts).

Modifications to an existing algorithm

Several modifications have been made upon the standard Minimax algorithm. First, we use alpha-beta pruning to cut off useless branches and reduce search cost. Second, we use iterative deepening by increasing the depth by one each time to the current limit. This gives us a stable best action at each stage. Third, to avoid being shut down and losing on timeout, the agent uses a simple time-emergency mechanism. Before starting deeper searching, it computes a safe fallback action using a one-step evaluation (choose_emergency_action). During search, the agent checks a per-move remaining time and stops cleanly if the limit is reached, returning the best action found so far. In which case, the agent can obtain better actions from deeper searching when time allows, while always having a legal and reasonable action ready when the time-used-so-far is elapsing the time constraint. These modifications, along with the optimizations we will discuss in the Optimization section, improve the overall efficiency of the search and the final performance of the algorithm, according to the game rules and constraints.

Explanation for the evaluation function

We use an evaluation function, which is a weighted feature-based model, to evaluate the board based on several aspects. Usually, a higher score means a better board for our player. The evaluation function final version of our agent mainly uses feature1_stack_num_diff (get_f1_score), feature2_stack_height_diff (get_f2_score), feature4_eat_diff (get_f4_score), feature5_cascade_diff (get_f5_score), feature6_same_line_diff (get_f6_score), and feature10_attackable_closest_dist (get_f10_score), according to computational studies on choosing different sets for the agents and evaluate the agent’s performances.
The final version of linear evaluation function:

score = f1_weight * feature1_opponent_remain
+ f2_weight * feature2_total_stack_height_diff
+ f4_weight * feature4_eat_diff
+ f5_weight * feature5_cascade_diff
+ f6_weight * feature6_same_line_diff
+ f10_weight * feature10_attackable_closest_dist

The first feature “opponents remain” is the number of stacks on the board. According to the tactical characteristics of the actions and the winning game conditions, this reflects a basic goal: eliminating the opponent stack.

The second feature “stacks total height diff ” is the difference in the overall stack heights between the two players. Generally, having higher stack heights implies more possible piece generations and more ways to apply pressure in terms of attacking. At the same time, it is safer in terms of being attacked by the opponent using EAT action in terms of defensive potential.

The fourth feature (immediate) “eat diff” is the difference in direct EAT opportunities. This measures how many immediate EAT options each side has. The fifth feature (immediate and foreseeing) “cascade diff” is the difference in direct meaningful CASCADE opportunities. We include this because cascade is a special and important action in Cascade, and a meaningful cascade can create pressure, push stacks toward the edge, or quickly change the local structure. The sixth feature (foreseeing) “same line diff” is the difference in meaningful same-line pressure. This measures whether our stacks are aligned against the opponent in a way that may lead to future attacks, and whether the opponent stacks are pressuring us using the same way. We are using the “diff” since, we are balancing the favorability between aggressively attacking the opponent stacks and conservatively protecting our own stacks. The main idea is: if our side can have more attackabilities while breaking the possible attacks from the opponent, this specific new state, and its corresponding action leading to it, are preferred.

The last important feature “attackable closest dist” measures the distance of our stacks to the closest attackable opponent stack. This encourages meaningful chasing (in a more future-promising sense), rather than only becoming closer in a general sense.

Using these features, the evaluation function rewards both immediate gains and future tactical opportunities.

We also experiment with mobility (feature3_legal_action_diff), edge-distance safety (feature7_average_edge_dist_diff), and repeated-state penalty (feature8_has_seen_penalty), but they are not directly included in the final linear evaluation function. The repeated-state penalty is still useful in our overall play-phase design, but we treat it mainly as part of search control and action optimisation, rather than as a core feature in the main evaluation function.

Learning weights for the Evaluation function

We use self-play as a tool to improve the weights used in the evaluation function in the play phase. Since our Minimax agent relies heavily on the weighted feature model in evaluate_new(...), the quality of these weights has a heavy influence on action selection. Instead of adjusting them only by intuition, we compare different weight settings by running repeated games between the same agent with different weights. In each experiment, one side uses weight set A and the other side uses weight set B, runs the same agent, and we swap the playing order so that both settings were tested as Red and Blue. This is necessary because the move order may influence the final results.

To do this, we build a batch self-play workflow that runs multiple games and records summary statistics such as win rate, relative strength between two weight sets, draw rate, and average total number of turns.

However, self-play also has some limitations. Since both sides are based on the same underlying Minimax framework, self-play mainly tells us which weight setting is stronger relative to another version of our own agent, rather than proving that the agent is globally strong against all styles of opponents. In addition, when two versions are very similar, many games may end in draws, which makes improvement harder to observe clearly. Therefore, we treat self-play as a practical tuning method for internal comparison.

Performance Evaluation

Other programs (agents) we have implemented

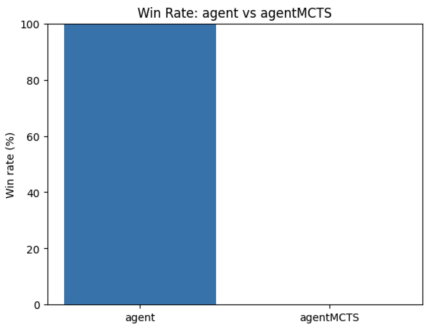
An agent powered by MCTS was also built, with full implementation of the key points. We implement selection with UCB1 score; expansion with ordered actions; simulation with single-step-forward evaluation and reasonable playout rewarding; and back-propagation. Namely, “agentMCTS”.

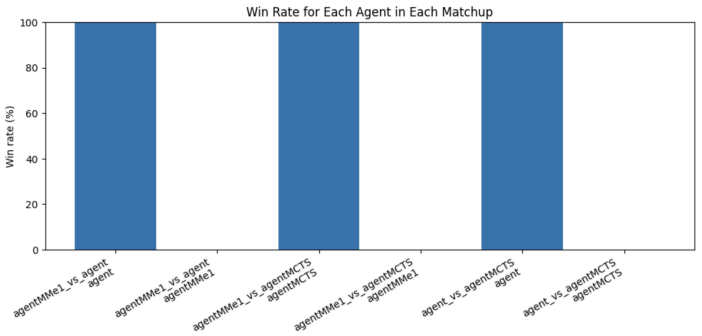
An agent powered by Minimax with alpha-beta pruning with a simple evaluation function (only two features). Namely, “agentMMe1”.

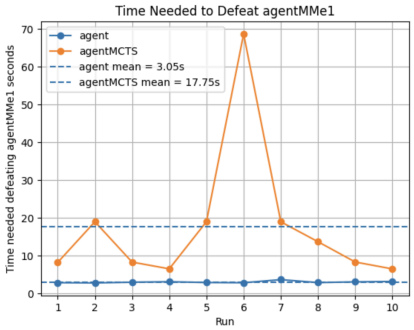
Reasons for choosing our algorithm (optimized MINIMAX rather than relatively plain MCTS)

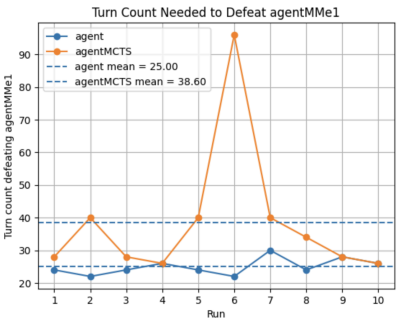
We regard the simple EVAL powered Minimax with alpha-beta agent as our base line, and compare the performance of agent and agentMCTS in terms of the turn number they reach a win, the time they take to win, and the probability of winning. (Run these two experiments and improve the feature or/and algorithm)

Experiment 1, let BLUE = agent/agentMCTS. Play 10 games against RED = agentMMe1; Play 5 games with RED = agent, BLUE = agentMCTS. The result is shown in the following graphs and table:

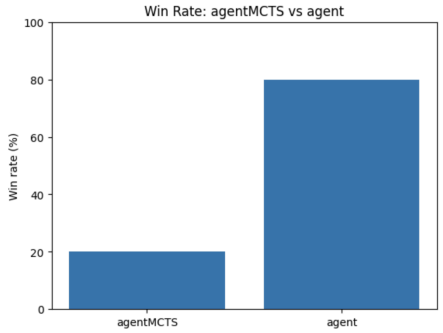


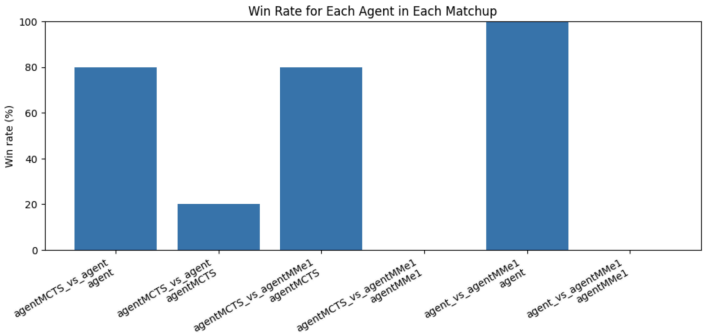


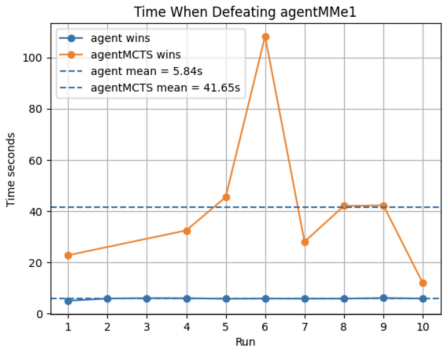


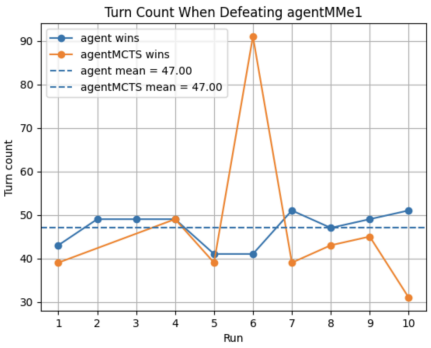


Experiment 2, let RED = agent/agentMCTS. Play 10 games against BLUE = agentMMe1; Play 5 games with RED = agentMCTS, BLUE = agent.









According to the computational studies about their performances compared to the baseline, MCTS does not show a clear outstandingness regarding Minimax, our optimized Minimax performs better most of the time. Moreover, it's using more time to win the game, and it has the risk of breaking the time limit.
Overall, Minimax has been shown to be more stable and easier to tune especially during the final winning period (e.g., for eliminating the final opponent stack when we are taking advantage). By contrast, we find that the MCTS agent gets harder to make reliable decisions in practice since we are using a simpler version of EVAL so as to satisfy the time limitation constraint and the playout does not behave well for cases like: balanced, final elimination chasing, etc. It still depends heavily on the effectiveness of evaluations during both expansion and simulation. So, as a result, we stick to the Minimax, because the MCTS does not perform better than the Minimax as we are expecting.
Therefore, our decision is to use the optimized iterative deepening Minimax with alpha-beta pruning as our final agent.

Limitations of our chosen algorithm

There are several limitations in the current play-phase strategy. First, our Minimax search is depth-limited to 4, so it can only capture relatively short tactical sequences. In Cascade, some important consequences of a move, especially those involving multi-step CASCADE pressure, repeated repositioning, or delayed edge elimination, may only become visible after more turns. As a result, the agent may choose an action that looks strong now but is weaker in the longer run. Alpha-beta pruning improves efficiency, but not this limitation.
Second, our evaluation function strongly depends on the features and weights we choose. Although the current features have captured several important factors, they cannot fully describe all of the board patterns. In particular, a linear weighted function does not handle repetition, chasing, or long-term trapping very well. Moreover, the weight adjustment based on the pattern is still quite simple and only based on the given root state, so different strategic signals may conflict with each other and lead to unstable preferences in some board configurations (especially as we search deeper, however, not that bad in our case since we just search till depth = 4).
Third, while we filter out meaningless cascade actions and move actions to reduce the branching factor and explore the most-preferred action first, our action ordering and pruning logic are still limited since there are so many possible states in the world. The overall search can still be expensive because the play phase may generate many legal successors, especially when both sides have multiple active stacks. And at the same time we are adding computational expenses for the logic checking for improving the algorithm behaviours.
Finally, we haven't taken space optimization into consideration. A possible further enhancement might be keeping track of the space that has been used during the implementation of the searching algorithms, so as to adjust the weight of the evaluation function or obtain a better action ordering.

Important Enhancements

Board state detect

To make the play-phase strategy more adaptive, we do not use the same feature weights for every search, given different boards (root state). Instead, we detect several board states and adjust the relative importance of features accordingly. The main detected situations include COMPACT_ALIGNMENT, OPPONENT_SCATTERED, EDGE_CORNER_PRESSURE, PLAYER_SCARCITY, OPPONENT_SCARCITY, ATTACKABLE_OPPONENT_FAR, and BoardState.HAS_IMMEDIATE_AFFECT. These states are used to reflect the fact that different board structures require different strategic priorities. For example, when edge or corner pressure exists, cascade-related and same-line pressures become more important because pushing stacks off the board is more promising. When our own stacks are scarce, stack-number-related features become more important because each remaining stack has higher strategic value. When attackable opponent stacks are still far away and especially when the board is balanced or our agent is taking advantage, the evaluation puts more emphasis on pressure and chasing-related signals, since the main goal is to approach the opponent efficiently in this case.
In addition, some detected board states are used not only in the evaluation function, but also in the search optimization stage. In BoardState.BALANCED positions, or in positions marked as BoardState.IN_OUR_FAVOUR, we guide the search more carefully by reordering legal actions with order_actions(...) and filtering strategically meaningless actions with filter_meaningful_actions(...). For example, we remove useless cascades through is_meaningless_cascade(...), and in balanced situations we may

also filter out movements that only increase the general distance from the opponent through is_meaningless_movement_balanced(...). This is especially useful in chasing situations, where the agent is encouraged to move toward opponent stacks that are realistically attackable.

Filtering and ordering actions before actual expansion

We filter and reorder legal actions before actual expansion. This is implemented in optimization.py, mainly through filter_meaningful_actions(...) and order_actions(...). The purpose is to reduce unnecessary search and make alpha-beta pruning more effective. We filter out meaningless CASCADE actions through is_meaningless_cascade(...), and in balanced positions we also filter out meaningless MOVE actions through is_meaningless_movement_balanced(...). These kinds of movements are treated as meaningless as they increase the general distance from the opponent but do not help our agent approach more effectively.

After filtering, we assign each remaining legal action a heuristic score and sort the actions before expanding them in Minimax. The base score comes from evaluate_new(...) on the resulting state. On top of this, we add tactical priorities: direct EAT actions get the highest bonus, CASCADE actions are preferred when they reduce the opponent's stack count, and MOVE actions are preferred when they reduce the distance to an attackable opponent stack. In this way, the action ordering is guided by both evaluation and tactical heuristics.

We also apply this optimization not only at the MAX level, but also at the MIN level of Minimax. This is important because Minimax assumes that the opponent also plays optimally. Therefore, ordering and filtering actions for the opponent side makes the search model more realistic and usually makes our own agent stronger in practice.

Penalty for seen states and move to stronger enemy

We also use two penalties in search. The first one is related to the threefold repetition rule, especially in chasing or balanced situations where repeated states may lead to a draw. It is computed by get_f8_score(...) from the states that have been seen recorded along the current search branch. The second one is a safety penalty. If an action moves our stack next to a stronger opponent stack or onto cascade-eliminatable coordinates, which are detected by moves_next_to_stronger_opponent(...) and moves_allow_direct_cascade_elimination (...), that action is also penalised.

Supporting Work

To support the development and evaluation of our game-playing agents, we implement a computational study framework that executes fixed runs of controlled matches between different agent versions and records their performance statistics. A custom Python driver is developed using subprocess to automatically invoke the referee for multiple matchups, including comparisons between the agent (optimized Minimax), the agentMCTS, and simple Minimax agent. For every run, the framework automatically extracts the winning agent, the final turn count of the last completed board state, the total execution time, and the full referee log. The results are stored in structured CSV files for further analysis.

To analyse and compare the performances of agents, a separate local analysis program is developed to generate comparative performance plots using the collected results. Two main line graphs are used for decision-making. The first graph plots the turn count required to defeat a baseline agent across repeated trials between different search strategies. The second graph plots the play time required to reach the victories between different search strategies against the baseline agent. These two graphs help measure algorithmic/strategic efficiency, as lower values generally indicate that an agent can convert advantageous board states into victories more efficiently.

In addition, win-rate statistics were generated for each matchup to measure robustness and consistency across repeated games, including cases where an agent failed to defeat its opponent. Finding the corresponding log file for the losing cases and analyzing the failing reason is essential for our EVAL refinement and algorithm optimization and modification enhancement during the development.

Conclusion

In summary, during the entire process of development we designed and set-up 3 different agents, using one as a basement (Minimax with simple EVAL) and kept adjusting and modifying the "agent" (optimized Minimax) and "agentMCTS" to improve their behaviors during the play phase. We have successfully improved the efficiency of the agent performance by implementing board detection, before-application action ordering, and penalty for disappointing states. However, there are still some downsides, specifically, getting trapped in chasing the opponents (ending in a draw even though our agent is taking advantage in the current situation).